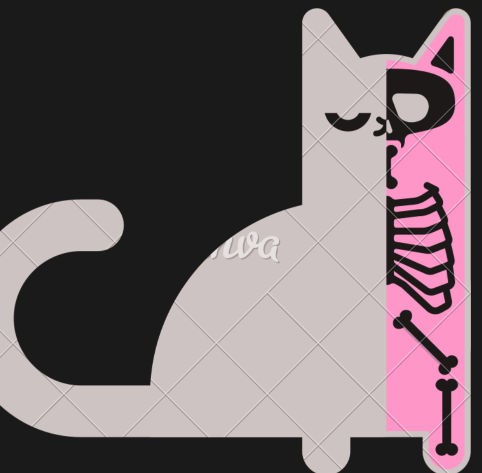
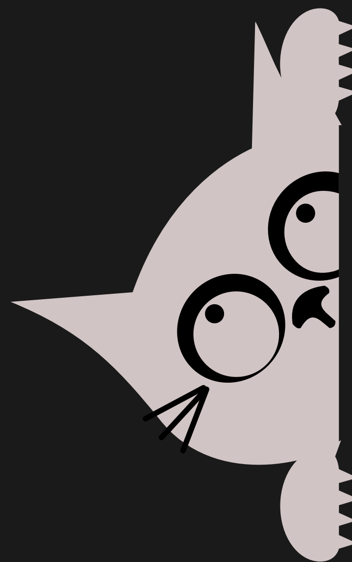
DESIGN OF QUANTUM CIRCUITS FOR IMPLEMENTING THE DEUTSCH-JOZSA ALGORITHM

AT ROOM TEMPERATURE

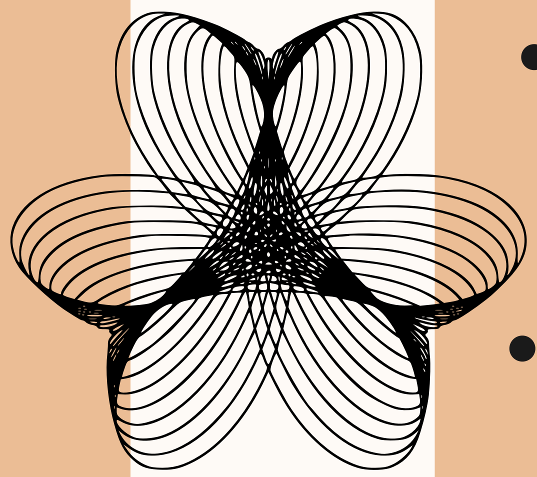




CLASSICAL COMPUTER VS QUANTUM COMPUTER

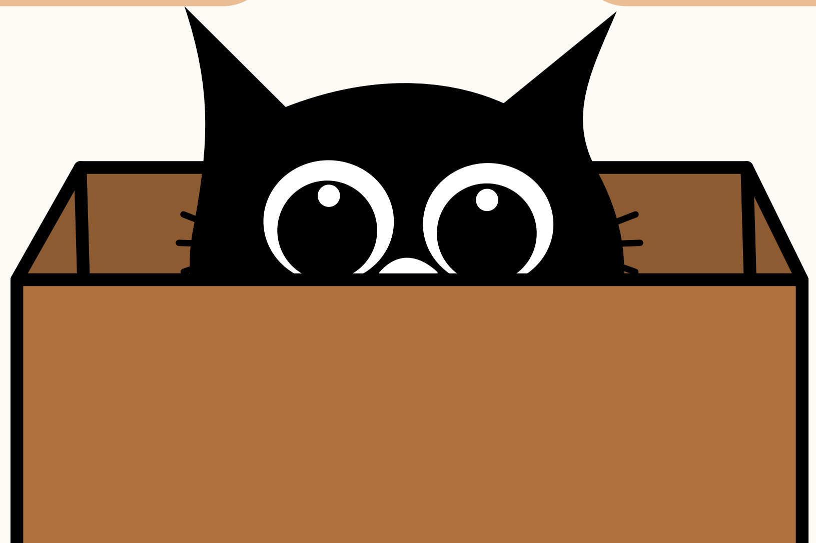
CLASSICAL COMPUTATION

- Evaluates function inputs **one at a time**.
- **Limitation:** For some problems (like Deutsch–Jozsa), may require **exponentially many evaluations**.



QUANTUM COMPUTATION

- Uses **superposition** and **entanglement** to evaluate **all inputs simultaneously**
- **Advantage:** Can solve specific problems **much faster** than classical computers.



IMPLEMENTATION OF DEUTSCH-JOZSA ALGORITHM

Determines if a function (f(x)) is **constant** or **balanced.**

Classical computers may need many function calls → $2^{(n-1)}$ +1 queries in the worst case

Quantum computer just **one query.**

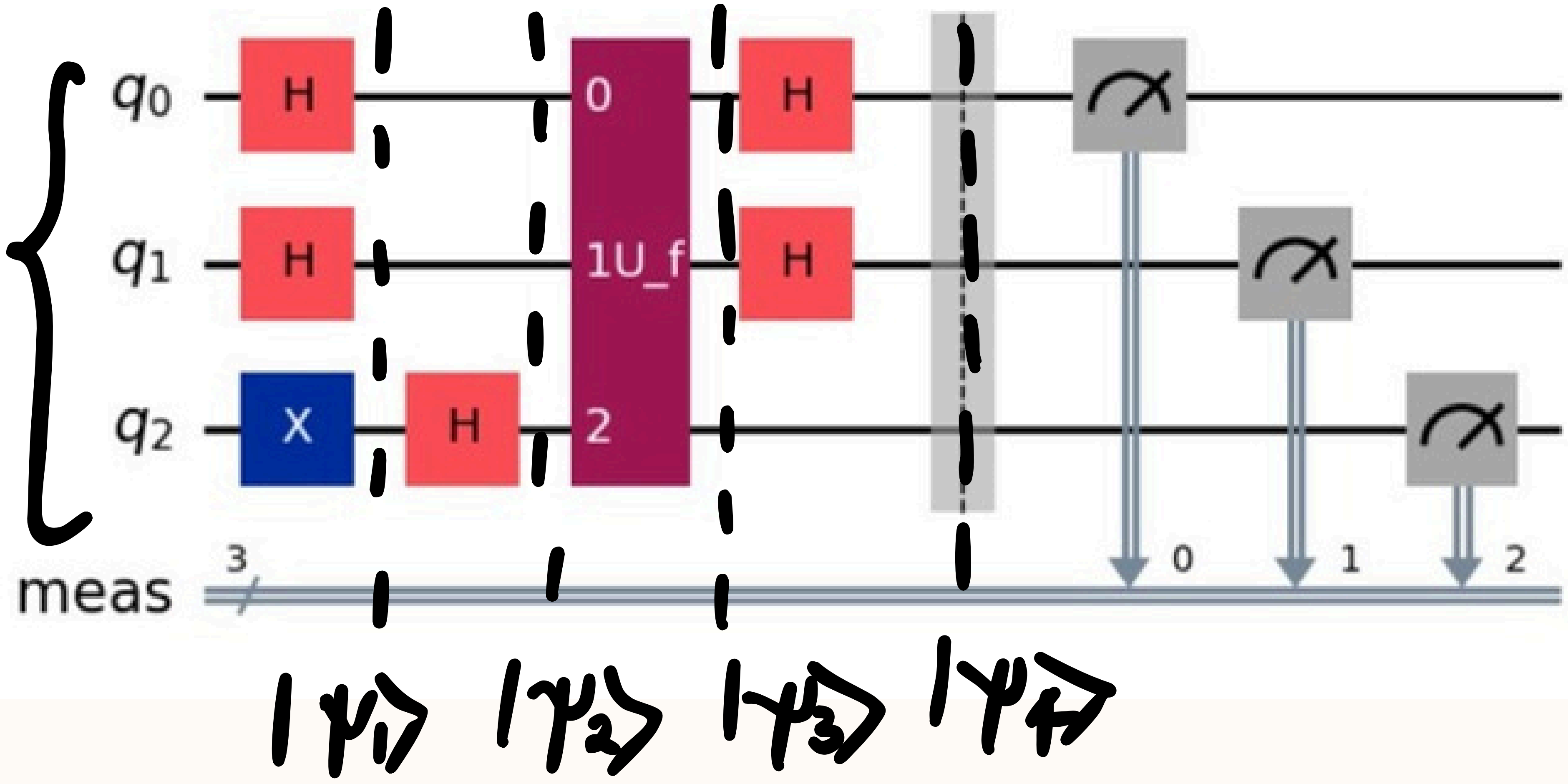
Steps:

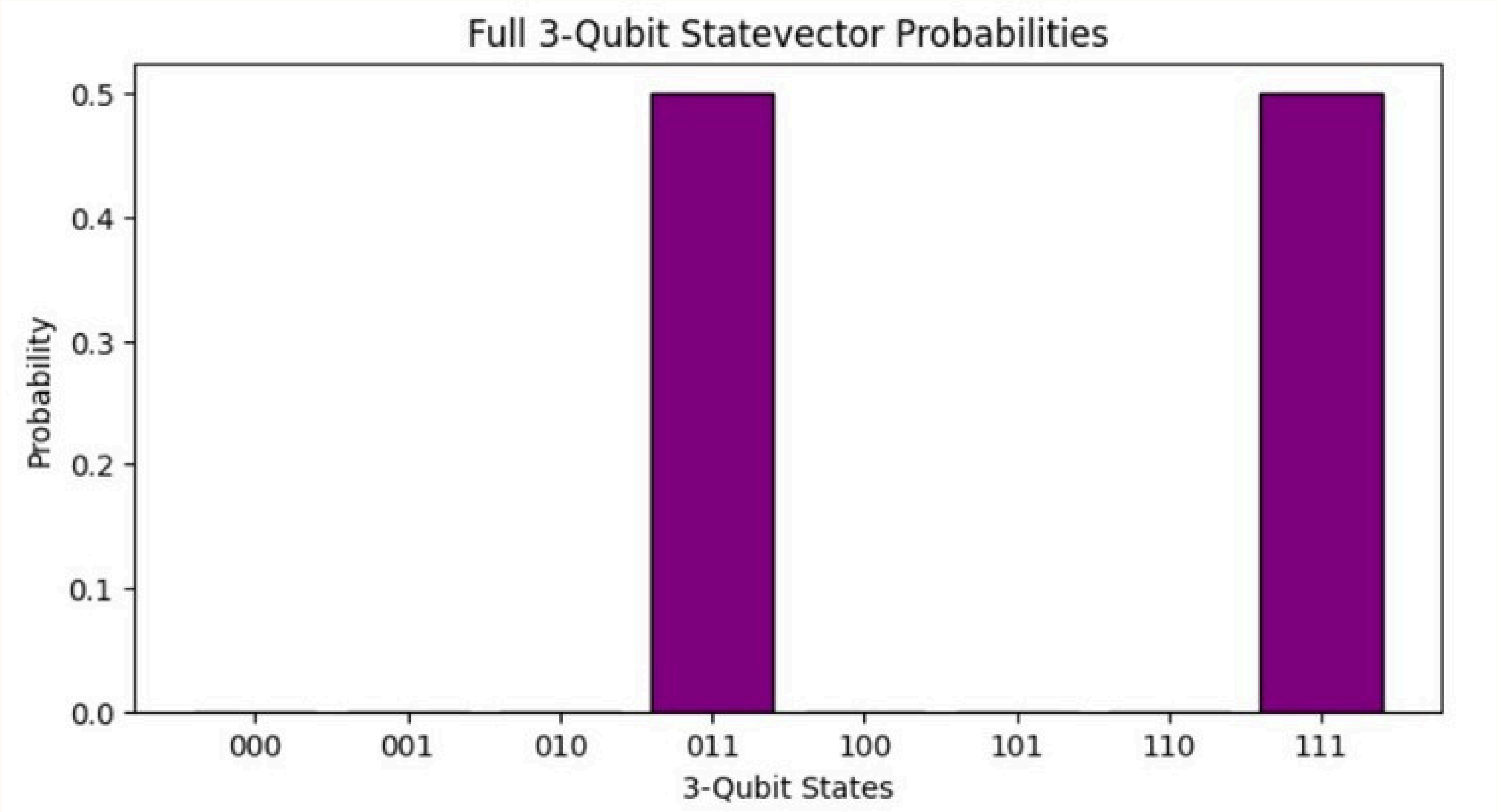
1. Initialize qubits ($|0\rangle^{\otimes n} |1\rangle$)
2. Apply Hadamard gates (H) to all qubits.
3. Apply the oracle Uf that flips the phase depending on f(x)f(x)f(x).
4. Apply Hadamard gates again to the first nnn qubits
5. If the result is all zeros → function is constant.

Otherwise → balanced.

IMPLEMENTATION

* $q_0, q_1 \rightarrow$ input qubits ; $q_2 \rightarrow$ output qubit used by Oracle function
* 3 qubits involved $\Rightarrow$ 8 possible combination





H-GATE $\rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$

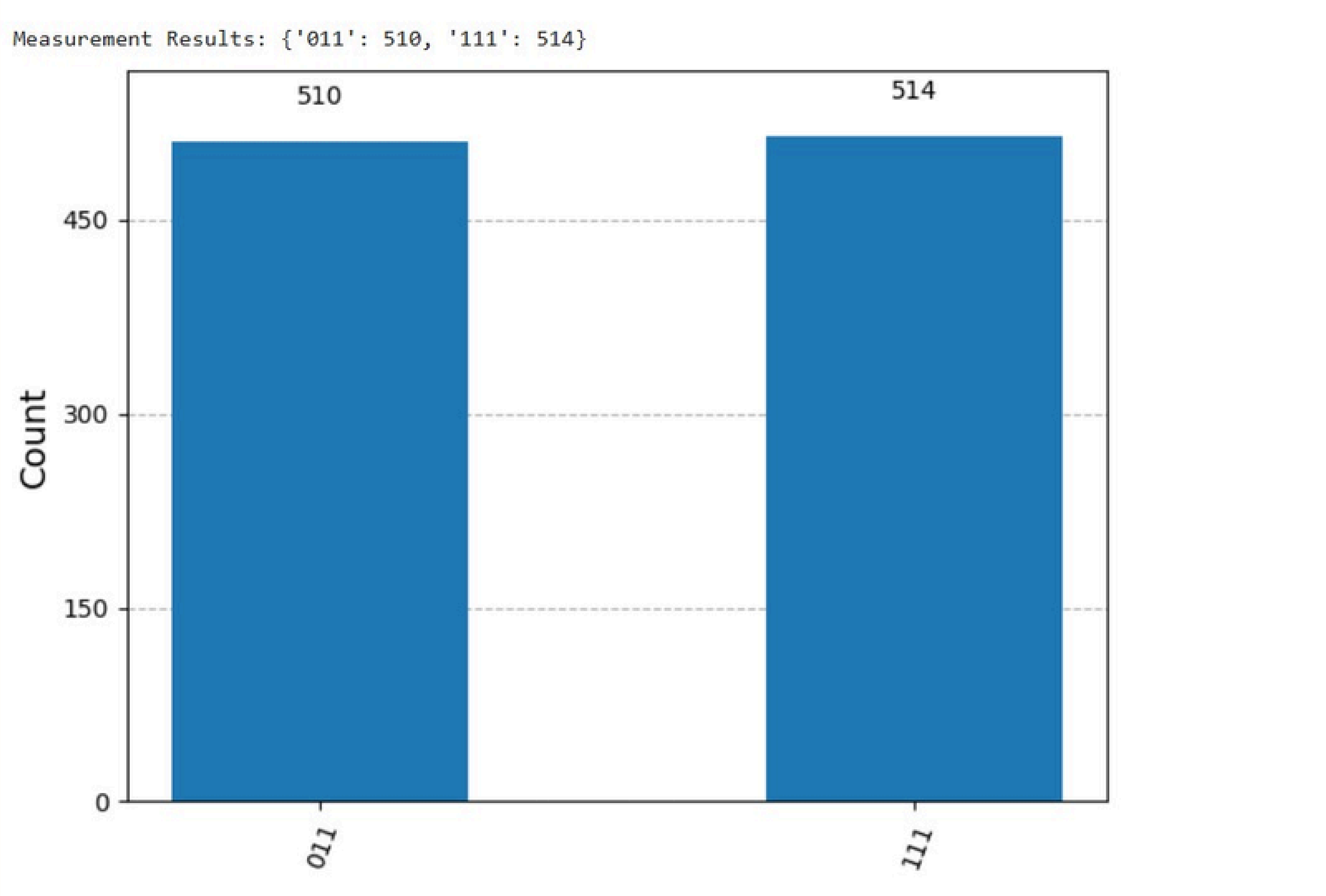
X-Gate $\rightarrow$ NOT$(|0\rangle)$; NOT$(|1\rangle)$

Oracle function :

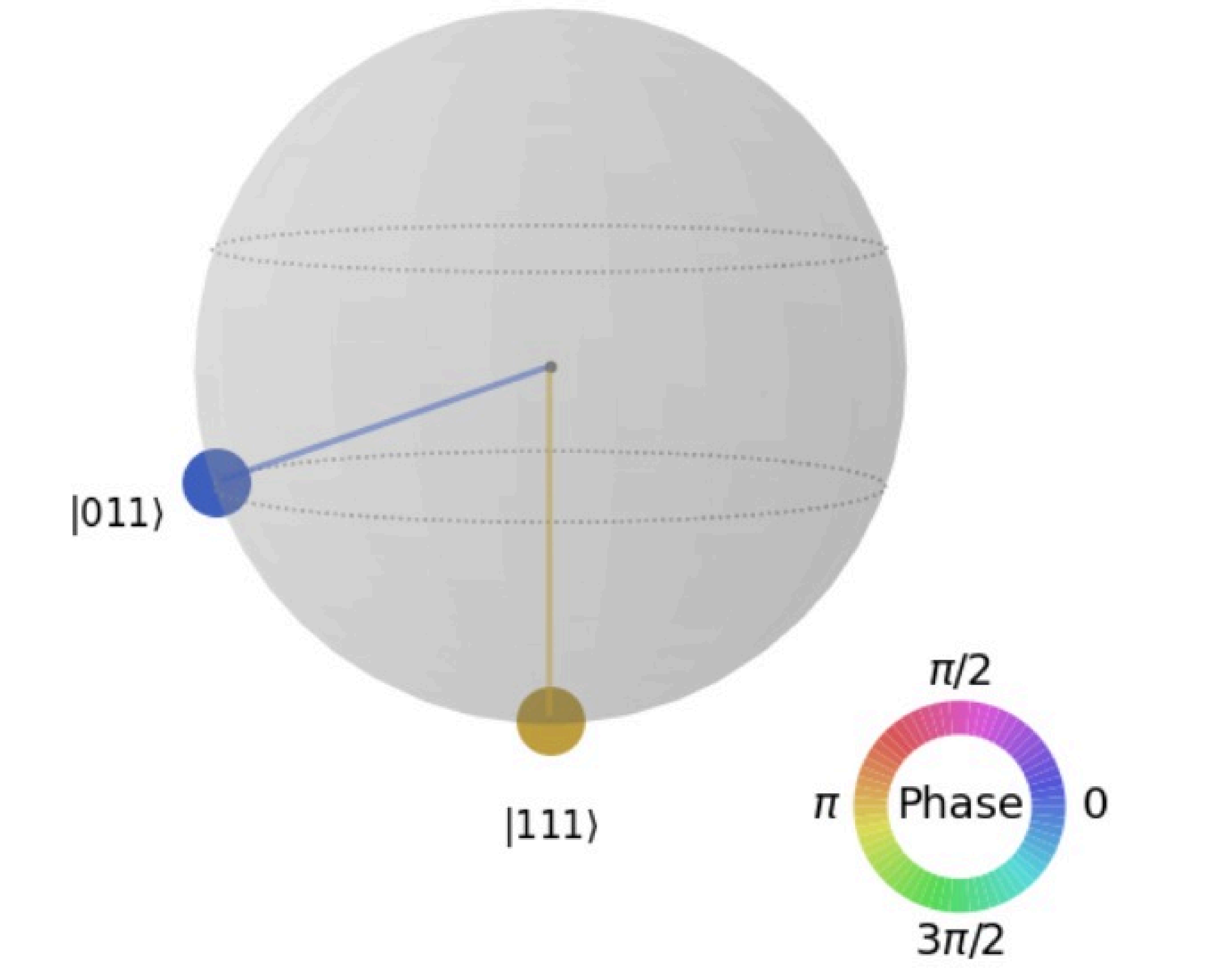
$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$$

RESULTS

Graphical representation



Bloch-Sphere representation



APPENDIX : QISKIT CODE

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram,
plot_state_city, plot_state_qsphere,
plot_bloch_multivector
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from qiskit.quantum_info import Statevector,
partial_trace, DensityMatrix

n = 3

# 🔹 Define Oracle
def deutsch_jozsa_oracle(case='balanced'):
    oracle = QuantumCircuit(3)
    if case == 'balanced':
        oracle.cx(0, 2)
        oracle.cx(1, 2)
    elif case == 'constant':
        pass
    return oracle
```

```
# Deutsch–Jozsa Circuit (without measurement for
state analysis)
def deutsch_jozsa_circuit_no_measure(oracle):
    dj = QuantumCircuit(3)
    dj.x(2)
    dj.h([0, 1, 2])
    dj.append(oracle.to_gate(label="U_f"), [0, 1, 2])
    dj.h([0, 1])
    return dj

# Deutsch–Jozsa Circuit (with measurement for
simulation)
def deutsch_jozsa_circuit(oracle):
    dj = deutsch_jozsa_circuit_no_measure(oracle)
    dj.measure_all()
    return dj

# Choose oracle
oracle = deutsch_jozsa_oracle('balanced')   #
'constant' or 'balanced'
dj_circuit = deutsch_jozsa_circuit(oracle)

print("Deutsch–Jozsa Circuit:")
display(dj_circuit.draw('mpl'))
```

WHY ROOM TEMPERATURE MATTERS?

Most quantum computers require extremely low temperatures (close to absolute zero).

Makes small-scale quantum experiments easier and practical

NMR qubits stay coherent even without cryogenics

Demonstrates quantum parallelism, since the algorithm evaluates the function on all inputs simultaneously using superposition.

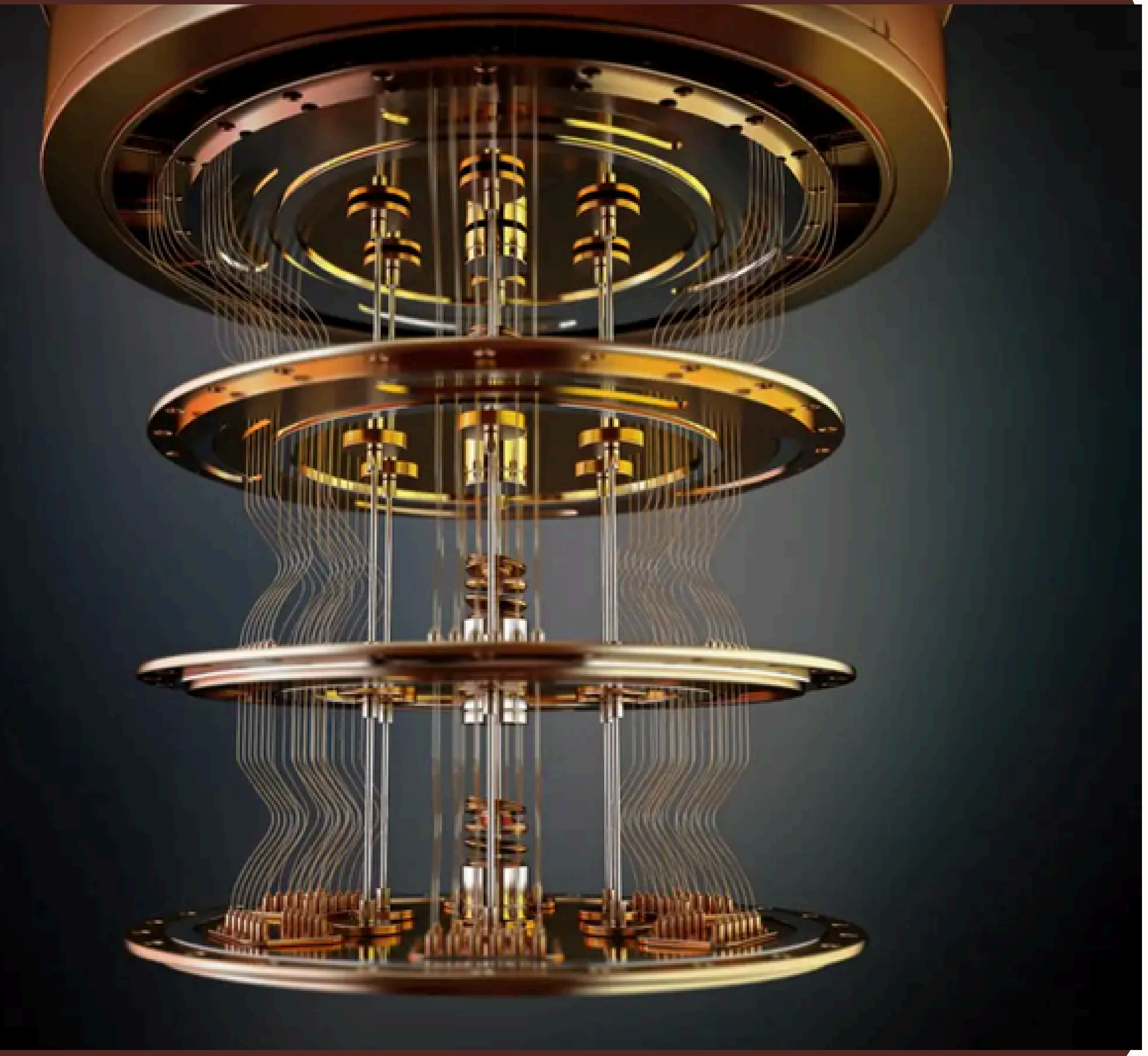
Room-temperature platforms like NMR and NV centers can:

Operate in a normal lab environment

Maintain stable quantum states for computation

Avoid expensive and complex cooling equipment

CONCLUSION



- Successfully implemented Deutsch–Jozsa algorithm using Qiskit simulator.
- Histogram results clearly distinguish constant vs balanced functions.
- Demonstrates quantum parallelism using superposition of all inputs.
- Fully digital implementation — no physical hardware or NMR required.
- Qiskit allows easy simulation and visualization of quantum algorithms.
- Highlights how simple quantum algorithms can be tested and understood before moving to real quantum hardware.